

SOP 31 — Match Ticks (evento ao vivo)

Camada A.N.I.: Instruments-Tools (emit_match_tick.py + poll_match) · Versão: 1.0 (Protocol 0, 2026-09-09)

Golden Rule: mudou a lógica → atualiza o SOP → só então o código.

Objetivo

Em dia de jogo do `profissional_m`, observar o estado da partida pela API de futebol e, a cada **mudança de estado**, emitir um `DeliveryPayload.MatchTick` — **sem LLM e sem aprovação humana** (vêm da API, não da IA). Ticks publicam direto; nunca inventar gol, cartão ou escalação.

Quando roda

- Worker paralelo em matchday: de **kickoff–90min** até **FT+30**, polling a cada **30s** (blueprint).
- Modo matchday: existe jogo `profissional_m` nas próximas 18 horas (SOP 30).

Input

- Estado atual do jogo via provedor de futebol (mesmo adapter do SOP 11), em modo mock quando não houver key (fixtures).
- Estado anterior conhecido: `.tmp/matches/<match_id>.json` (último poll) — a transição é o que gera tick.

Output

- `DeliveryPayload.MatchTick` para cada evento detectado → `.tmp/ticks/<match_id>-<AAAAMDDTH-HMM>.json` + publicação imediata no destino de ticks (webhook do editor/home de jogo) com `publish_without_approval: true`.

Passos determinísticos

1. Calcular janela ativa (kickoff–90min ... FT+30); fora dela → dormir até o próximo agendamento (sem loop infinito dentro da tool — o scheduler controla).
2. A cada poll (30s): buscar estado (minuto, placar, eventos: gol/cartão/escalação confirmada/kickoff/intervalo/fim).
3. Comparar com o último estado conhecido:
 - Mudou `score` → tick `type: "goal"` (ou cartão/etc. conforme evento da API), `minute` do evento, `score` novo no formato `"1-0"`, `text` curto em pt-BR (ex.: “Gol do Flamengo. Pedro, 23’.” — texto montado a partir dos dados da API, não de LLM).
 - `lineup_confirmed` passou para true → tick `type: "lineup"` (escalação oficial do provedor).
 - Kickoff/intervalo/fim de jogo → ticks `kickoff / ht / ft`.
4. **Anti-duplicidade:** só emite tick quando há transição real de estado (mesmo `minute+type+score` já emitido → ignora).
5. Gravar o tick com `match_id`, `minute`, `type`, `text`, `score`, `publish_without_approval: true` e publicar (sem fila humana).

6. Atualizar `.tmp/matches/<match_id>.json` com o novo estado.
7. FT+30 → encerrar worker e registrar fim de jogo no `editor_notes` do próximo payload.

Edge cases

- API retorna o mesmo estado 2 polls seguidos → nenhum tick (transição é o evento).
- API com lag (gol aparece com atraso) → o tick sai quando a API mostra; nunca “adivinhar” minuto.
- Placar regredindo (correção da API) → emitir tick apenas se a API confirmar correção (ex.: tipo `ft` / nota); dúvida → não emite, loga.
- API fora do ar durante o jogo → 3 retries com backoff; persistindo, marcar jogo como `degraded` em `progress.md` (nunca fabricar tick).
- Sem key (mock) → testes com fixtures apenas; worker real exige key.
- Escalação: só com `lineup_confirmed: true` vindo do provedor.

Rate limits / limites conhecidos

- Polling a cada 30s (nunca menos); respeitar o limite diário de requests do plano do provedor (calcular: $\sim 3 \text{ req/min} \times \text{duração do jogo}$).
- Ticks publicados via webhook do editor: respeitar quota do Slack/Discord (ver SOP 40).

Testes

- Fixtures: `fixtures/match_scheduled.json` → nenhum tick (sem transição); `fixtures/match_live_goal.json` (estado live, 1-0) comparado com o estado anterior scheduled → exatamente **1 tick** de gol com `score: "1-0"` e `publish_without_approval: true`.
- Rodar o mesmo par 2x → segundo ciclo sem tick (anti-duplicidade).